

不可变组件 (immutable components)

1. 不可变组件介绍

不可变组件一旦插入到实体中，这些数据就无法被修改（例如：你不能获取不可变组件的可变借用 `&mut MyComponent`）。修改不可变组件的唯一方法是插入一个新的实例覆盖原来的。

2. 创建不可变组件

你可以通过为组件添加 `#[component(immutable)]` 属性来使其成为不可变组件：

```
#[derive(Component)]
#[component(immutable)]
pub struct MyComponent(pub u32);
```

3. 捕获不可变组件的每一次变化

通过遵循这一限制，我们可以确保组件的生命周期钩子和观察者（插入、移除）能够捕捉到发生的每一次变化，从而保证复杂不变量得以维持。

为了说明这一点，下面是一个示例：我们追踪所有 `SumMe` 组件的全局总和。

```
#[derive(Component)]
#[component(
    immutable,
    on_insert = add_when_inserting,
    on_remove = subtract_when_removing
)]
struct SumMe(u32);

// 在插入 SumMe 时将值加到 TotalSum 中
fn add_when_inserting(mut world: DeferredWorld, context: HookContext) {
    if let Some(sum_me) = world.get::<SumMe>(context.entity) {
        world.resource_mut::<TotalSum>().0 += sum_me.0;
    }
}

// 在移除 SumMe 时将值从 TotalSum 中减去
fn subtract_when_removing(mut world: DeferredWorld, context: HookContext) {
    if let Some(sum_me) = world.get::<SumMe>(context.entity) {
        world.resource_mut::<TotalSum>().0 -= sum_me.0;
    }
}

// 存储所有 SumMe 值的总和
#[derive(Resource)]
struct TotalSum(u32);

// 生成 SumMe
fn spawn_sum_me(mut commands: Commands) {
    commands.spawn(SumMe(5));
    commands.spawn(SumMe(3));
}

// 修改 SumMe
fn modify_sum_me(mut commands: Commands, query: Query<(Entity, &SumMe)>) {
    for (entity, sum_me) in query {
        commands.entity(entity).insert(SumMe(sum_me.0 + 1));
    }
}
```

4. 总结

除了不可变组件 + `OnInsert`、`OnRemove` 组件钩子或观察者，我们之前还学到了可以使用 `Changed` 查询过滤器捕获变化，只是 `Changed` 会产生延迟，无法在组件变化时立刻响应。需要注意，只能用整体替换的方式修改不可变组件，如果不可变组件经常被修改或不可变组件中有大量数据，那么这将严重降低游戏性能。

虽然不可变组件是一个相对小众的工具，但对于那些很少被修改（或数量很少）、对正确性要求极高的组件来说，它们是非常合适的选择。例如，`Bevy 0.16` 在其全新的关系系统中就使用了不可变组件（以及钩子！）作为基础，以确保关系的两端（例如 `ChildOf` 和 `Children`）能够保持同步。